

Java EE 框架整合开发入门到实战—— —Spring+Spring MVC+MyBatis（微 课版）

实验指导书

2019.11

目录

概 述.....	1
实验一 Java EE 开发环境.....	2
实验二 Bean 的装配（基于注解方式）	5
实验三 Spring 的事务管理（基于注解的事务管理）	6
实验四 MyBatis 与 Spring 的整合（映射器及动态 SQL）	11
实验五 Controller 接收请求参数（注册与登录系统）	15
实验六 表单与数据绑定（学生信息管理项目）	21
实验七 拦截器应用案例（登录权限控制）	28
实验八 JSR 303 验证（表单验证）	29
实验九 SSM 框架整合应用测试（学生信息管理项目）	30
实验十 在 Eclipse（或 STS 或 IDEA）中使用 Maven 整合 SSM 框架（学生信息管理项目）	31
实验十一 综合实验（电子商务平台的设计与实现）	34

Java EE 框架整合开发实验指导书 陈仁编写

概 述

本课程实验教学的目的在于训练学生的实际动手能力，以期更好地掌握 Java EE 相关原理、技术及方法，并为后期毕业设计以及以后从事 Java 相关开发打下坚实的基础。

实验内容

与教材《Java EE 框架整合开发入门到实战——Spring+Spring MVC+MyBatis（微课版）》对应的实验共有 11 个（共 38 学时），分别如下：

- 1、Java EE 开发环境 2 学时
- 2、Bean 的装配（基于注解方式） 2 学时
- 3、Spring 的事务管理（基于注解的事务管理） 2 学时
- 4、MyBatis 与 Spring 的整合（映射器及动态 SQL） 4 学时
- 5、Controller 接收请求参数（注册与登录系统） 2 学时
- 6、表单与数据绑定（学生信息管理项目） 2 学时
- 7、拦截器应用案例（登录权限控制） 2 学时
- 8、JSR 303 验证（表单验证） 2 学时
- 9、SSM 框架整合应用测试（学生信息管理项目） 2 学时
- 10、在 Eclipse（或 STS 或 IDEA）中使用 Maven 整合 SSM 框架（学生信息管理项目） 2 学时
- 11、综合实验（电子商务平台的设计与实现） 16 学时

实验环境

硬件为个人微机，软件为操作系统 Windows 7 或 10，开发环境为 Eclipse-jee（或 STS 或 IDEA）+ JDK8（或以上）+ Tomcat9.0 + MySQL 5.5。

实验要求

本课程实验教学的要求包括：

- 1、根据教材中规定的内容在要求的时间内完成实验内容；
- 2、成功部署实验中要求实现的内容，并能演示；
- 3、提交实验中规定的开发内容的核心代码；
- 4、提交实验报告。

实验一 Java EE 开发环境

（实验课时：2 实验性质：设计）

实验名称：

Java EE 开发环境

实验目的和要求：

- 1、掌握基于 Eclipse 平台（或 STS 或 IDEA）的 Java EE 集成开发环境的构建。
- 2、通过在 Java EE 开发环境中创建和运行一些实例项目，熟悉 Java EE 的基本开发、部署和运行过程，为后续实验打下基础。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

1、安装并配置 JDK

按照提示安装完成 JDK 后，需要配置“环境变量”的“系统变量”Java_Home 和 Path。

2、安装 Tomcat

将下载的 apache-tomcat-9.0.2-windows-x64.zip 解压到某个目录下，比如解压到 E:\Java soft，解压缩即完成安装。

3、安装 Eclipse

Eclipse 下载完成后，解压缩到自己设置的路径下，即完成安装。

4、集成 Tomcat

启动 Eclipse，选择“Window”->“Preferences”菜单项，在弹出的对话框中选择“Server”->“Runtime Environments”命令。在弹出的窗口中，单击“Add”按钮，弹出如图 1 所示的“New Server Runtime Environment”界面，在此可以配置各种版本的 Web 服务器。

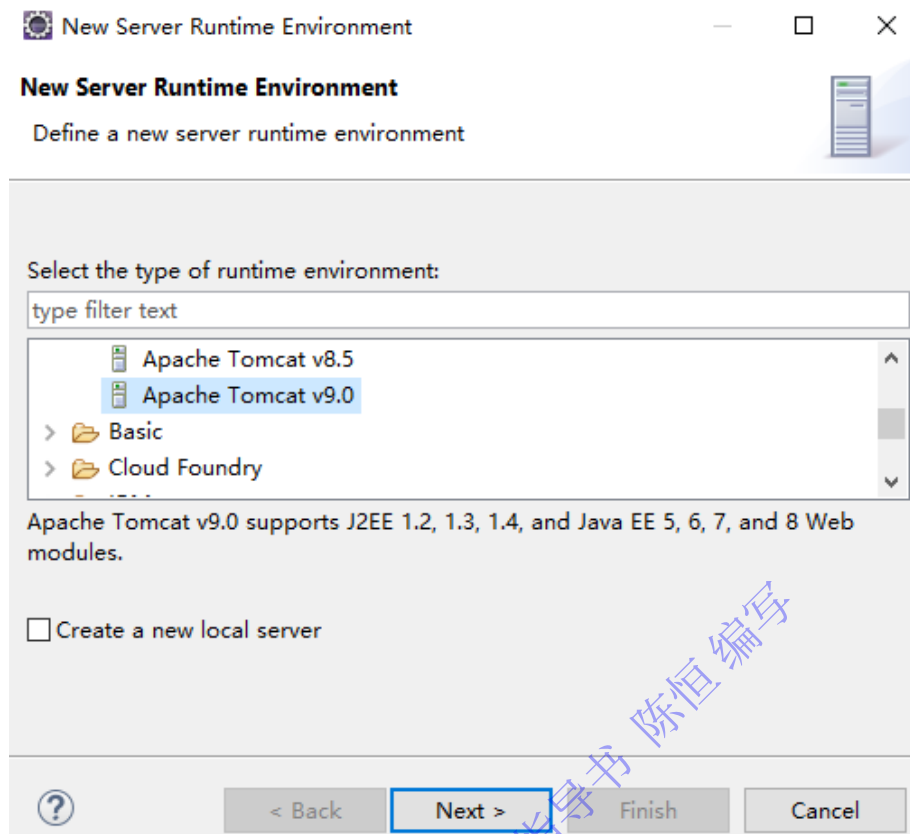


图 1 Tomcat 配置界面

在图 1 中选择“Apache Tomcat v9.0”服务器版本，单击“Next”按钮，进入如图 2 所示界面。

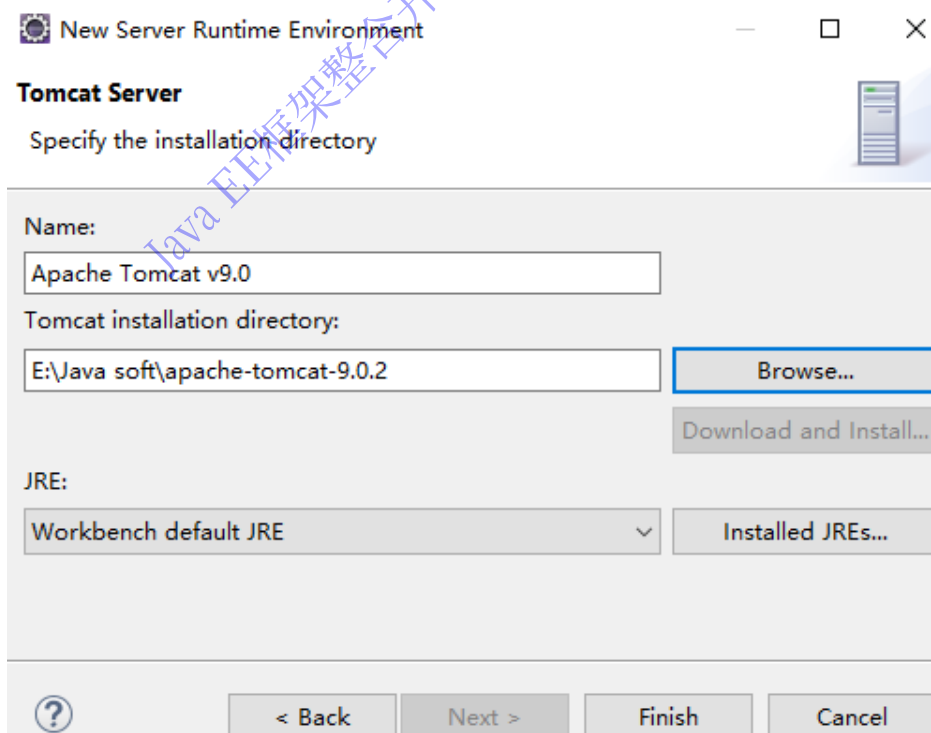


图 2 选择 Tomcat 目录

在图 2 中单击“Browse...”按钮，选择 Tomcat 的安装目录，单击“Finish”即可完成 Tomcat

配置。

至此，可以使用 Eclipse 创建 Dynamic Web project，并在 Tomcat 下运行。

5、下载 Spring

使用 Spring 框架开发应用程序时，除了引用 Spring 自身的 JAR 包外，还需要引用 commons.logging 的 JAR 包。

Spring 官方网站升级后，建议通过 Maven 和 Gradle 下载。对于不使用 Maven 和 Gradle 下载的开发人员，本实验给出一个 Spring Framework jar 官方直接下载路径：<http://repo.springsource.org/libs-release-local/org/springframework/spring/>。本实验采用的是 spring-framework-5.0.2.RELEASE-dist.zip。将下载到的 ZIP 文件解压缩，解压缩后的 libs 目录包含开发 Spring 应用所需要的 JAR 包（spring-XXX-5.0.2.RELEASE.jar）。

Spring 框架依赖于 Apache Commons Logging 组件，该组件的 JAR 包可以通过网址“http://commons.apache.org/proper/commons-logging/download_logging.cgi”下载，本实验下载的是“commons-logging-1.2-bin.zip”，解压缩后，即可找到“commons-logging-1.2.jar”。

对于 Spring 框架的初学者，开发 Spring 应用时，只需要将 Spring 的四个基础包（spring-core-5.0.2.RELEASE.jar、spring-beans-5.0.2.RELEASE.jar、spring-context-5.0.2.RELEASE.jar 和 spring-expression-5.0.2.RELEASE.jar）和 commons-logging-1.2.jar 复制到 Web 应用的 WEB-INF/lib 目录下即可。

6、开发一个简单的 Spring 程序

参考教材 1.3 节，使用 Eclipse 再开发一个简单的 Spring 程序，并测试运行。

实验二 Bean 的装配（基于注解方式）

（实验课时：2 实验性质：设计）

实验名称：

Bean 的装配（基于注解方式）

实验目的和要求：

- 1、掌握 Bean 的常用装配方式，尤其是基于注解的装配方式。
- 2、掌握 Spring 框架定义的一系列常用注解的使用方法，包括@Component、@Repository、@Service、@Controller 和@Autowired 等注解。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

1、使用@Repository 注解声明 DAO 层

参考 3.5.2 节，创建 Web 应用 practice2，并导入相关 JAR 包（参考实验一），在 practice2 应用的 src 中，创建 dao 包，该包下创建 MyDao 接口和 MyDaoImpl 实现类，并将实现类 MyDaoImpl 使用@Repository 注解标注为数据访问层。

2、使用@Service 注解声明 Service 层

在 practice2 应用的 src 中，创建 service 包，该包下创建 MyService 接口和 MyServiceImpl 实现类，并将实现类 MyServiceImpl 使用@Service 注解标注为业务逻辑层。

在 MyServiceImpl 类中使用@Autowired 注解装配 DAO 层声明的 Bean。

3、使用@Controller 注解声明控制器层

在 practice2 应用的 src 中，创建 controller 包，该包下创建 TestController 类，并将 MyController 类使用@Controller 注解标注为控制器层。

在 MyController 类中使用@Autowired 注解装配 Service 层声明的 Bean。

4、配置注解

在 practice2 应用的 src 中，创建配置文件 annotationContext.xml，并在该配置文件中，使用“<context:component-scan base-package="Bean 所在的包路径"/>”配置注解。

5、创建测试类

在 practice2 应用的 src 中，创建 test 包，并在该包中创建与运行测试类 MyTest。

实验三 Spring 的事务管理（基于注解的事务管理）

（实验课时：2 实验性质：设计）

实验名称：

Spring 的数据库编程（基于注解的事务处理）

实验目的和要求：

- 1、了解 Spring JDBC 的配置。
- 2、了解 Spring JdbcTemplate 的常用方法。
- 3、掌握基于 @Transactional 注解的声明式事务管理。
- 4、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

- 1、创建 Web 应用并导入 JAR 包

创建一个名为 practice3 的 Web 应用，将图 3 所示的 JAR 包复制到应用的 /WEB-INF/lib 目录。

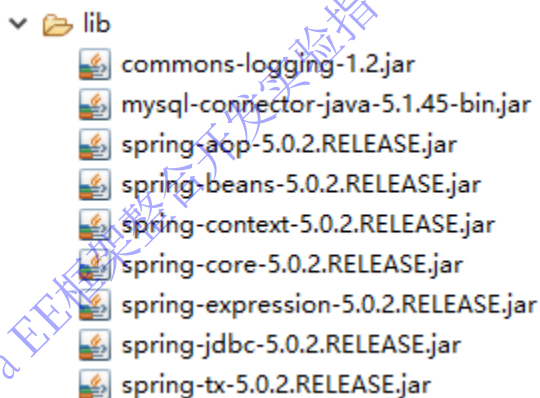


图 3 所需 JAR 包

- 2、创建并编辑配置文件

在 Web 应用 practice3 的 src 目录下，创建配置文件 applicationContext.xml，在该文件中配置数据源和 JDBC 模板，并使用 <tx:annotation-driven> 元素为事务管理器注册注解驱动器。

具体代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:tx="http://www.springframework.org/schema/tx"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
```



```

    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/tx
    http://www.springframework.org/schema/tx/spring-tx.xsd">
<!-- 指定需要扫描的包（包括子包），使注解生效 -->
<context:component-scan base-package="com"/>
<!-- 配置数据源 -->
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <!-- MySQL 数据库驱动 -->
    <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
    <!-- 连接数据库的 URL -->
    <property name="url" value="jdbc:mysql://localhost:3306/springtest?characterEncoding=utf8"/>
    <!-- 连接数据库的用户名 -->
    <property name="username" value="root"/>
    <!-- 连接数据库的密码 -->
    <property name="password" value="root"/>
</bean>
<!-- 配置 JDBC 模板 -->
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
</bean>
<!-- 为数据源添加事务管理器 -->
<bean id="txManager"
    class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource" />
</bean>
<!-- 为事务管理器注册注解驱动 -->
<tx:annotation-driven transaction-manager="txManager" />
</beans>

```

3、创建实体类

在 Web 应用 practice3 的 src 目录下，创建包 com.entity，在该包中创建实体类 MyUser。该类的属性与数据表 user 的字段一致。

```

package com.entity;

public class MyUser {
    private Integer uid;
    private String uname;
    private String usex;
    //此处省略 setter 和 getter 方法
    public String toString() {
        return "myUser [uid=" + uid + ", uname=" + uname + ", usex=" + usex + "];"
    }
}

```

在名为 springtest 的数据库中创建数据表 user，其结构如图 4 所示。

名	类型	长度	小数点	允许空值 (	
uid	int	10	0	☐	1
uname	varchar	20	0	☒	
usex	varchar	10	0	☒	

图 4 user 表的结构

4、创建数据访问层 Dao

在 Web 应用 practice3 的 src 目录下，创建名为 com.dao 的包，并在该包中，创建 UserDao 接口和 UserDaoImpl 实现类。在实现类 UserDaoImpl 中使用 JDBC 模块 JdbcTemplate 访问数据库（添加与查询用户），并将该类注解为@Repository("testDao")。

UserDao 接口的代码如下：

```
package com.dao;

public interface UserDao{

    public int save(String sql, Object param[]);

    public int delete(String sql, Object param[]);

}
```

UserDaoImpl 实现类的代码如下：

```
package com.dao;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
@Repository("userDao ")
public class UserDaoImpl implements UserDao {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Override
    public int save(String sql, Object[] param) {
        return jdbcTemplate.update(sql,param);
    }

    @Override
    public int delete(String sql, Object[] param) {
        return jdbcTemplate.update(sql,param);
    }

}
```

5、创建 Service 层

在 Web 应用 practice3 的 src 目录下，创建名为 com.service 的包，并在该包中创建 UserService 接口和 UserServiceImpl 实现类。在 Service 层依赖注入数据访问层，并添加@Transactional 注解进行事务管理。具体代码如下：

```

package com.service;

public interface UserService {

    public void test();

}

package com.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.dao.UserDao;

@Service("userService")
@Transactional
//加上注解@Transactional,就可以指定这个类需要受 Spring 的事务管理
//注意@Transactional 只能针对 public 属性范围内的方法添加
public class UserServiceImpl implements UserService{

    @Autowired
    private UserDao userDao;

    @Override
    public void test() {

        String deleteSql = "delete from user";
        String saveSql = "insert into user values(?,?,?)";
        Object param[] = {1,"chenheng","男"};
        userDao.delete(deleteSql, null);
        userDao.save(saveSql, param);
        //插入两条主键重复的数据
        userDao.save(saveSql, param);

    }

}

```

6、创建 Controller 层

在 Web 应用 practice3 的 src 目录下，创建名为 com.controller 的包，并在该包中创建 UserController 控制器类。在控制层依赖注入 Service 层，代码如下：

```

package com.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import com.service.UserService;

@Controller
public class UserController{

    @Autowired
    private UserService userService;

    public void test() {

        userService.test();

    }

}

```

7、创建测试类

在 Web 应用 practice3 的 src 目录下，创建名为 com.test 的包，并在该包中创建测试类 UserTest。在测试类中通过访问 Controller，测试基于注解的声明式事务管理。

测试类 UserTest 的代码如下：

```
package com.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.controller.UserController;

public class UserTest{

    public static void main(String[] args) {

        ApplicationContext appCon =
        new ClassPathXmlApplicationContext("applicationContext.xml");

        UserController uc= (UserController)appCon.getBean("userController");
        uc.test();

    }

}
```

8、运行测试类，测试事务管理

运行测试类，查看数据库插入两条 ID 相同的数据，事务管理是否好用。

实验四 MyBatis 与 Spring 的整合（映射器及动态 SQL）

（实验课时：4 实验性质：设计）

实验名称：

MyBatis 与 Spring 的整合（映射器及动态 SQL）

实验目的和要求：

- 1、掌握 MyBatis 与 Spring 框架的整合开发。
- 2、熟练掌握 MyBatis 的 SQL 映射文件的编写。
- 3、掌握 MyBatis 的动态 SQL 语句的拼接语法。
- 4、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

1、创建 Web 应用

创建 Web 应用 practice4。

2、导入相关的 JAR 包

参考教材的 6.5 节，导入 MyBatis 与 Spring 框架整合开发所需要的 JAR 包。包括 MyBatis 框架所需的 JAR 包，Spring 框架所需的 JAR 包，MyBatis 与 Spring 整合的中间 JAR 包，数据库驱动 JAR 包，数据源所需的 JAR 包。

3、创建实体类

在 Web 应用 practice4 的 src 中，创建名为 com.po 的包，并在该包中创建持久化实体类 MyUser。

4、创建映射文件

在 Web 应用 practice4 的 src 中，创建名为 com.mybatis 的包，并在该包中创建 SQL 映射文件 UserMapper.xml。在 SQL 映射文件中，实现如下功能的 SQL 映射：

- （1）根据 uid 查询一个用户信息
- （2）查询所有用户信息
- （3）添加一个用户
- （4）修改一个用户
- （5）删除一个用户
- （6）使用 foreach 元素，查询 id 在（1,3,5,7,9,11）中的用户信息

5、创建 MyBatis 核心配置文件

在该包 com.mybatis 中创建 MyBatis 核心配置文件 mybatis-config.xml，具体代码如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <!-- 告诉 MyBatis 到哪里去找映射文件 -->
    <mappers>
        <mapper resource="com/mybatis/UserMapper.xml"/>
    </mappers>
</configuration>
```

6、创建数据访问接口

在 Web 应用 practice4 的 src 中，创建名为 com.dao 的包，在该包中创建 UserDao 接口，并将接口使用 @Mapper 注解为 Mapper，接口中的方法与 SQL 映射文件中的 ID 一致。

7、创建日志文件

在 Web 应用 practice4 的 src 中，创建日志文件 log4j.properties，文件内容如下：

```
# Global logging configuration
log4j.rootLogger=ERROR, stdout
# MyBatis logging configuration...
log4j.logger.com.dao=DEBUG
# Console output...
log4j.appender.stdout=org.apache.log4j.ConsoleAppender
log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern=%5p [%t] - %m%n
```

8、创建控制层

在 Web 应用 practice4 的 src 中，创建一个名为 com.controller 的包，在该包中创建 UserController 类，在该类中调用数据访问接口中的方法。

9、创建 Spring 的配置文件

在 Web 应用 practice4 的 src 中，创建配置文件 applicationContext.xml。在配置文件中配置数据源、MyBatis 工厂以及 Mapper 代理开发等信息。具体代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
```

```

xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/tx
    http://www.springframework.org/schema/tx/spring-tx.xsd">
<!-- 指定需要扫描的包（包括子包），使注解生效 -->
<context:component-scan base-package="com.dao"/>
<context:component-scan base-package="com.controller"/>
<!-- 配置数据源 -->
<bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource">
    <property name="driverClassName" value="com.mysql.jdbc.Driver" />
    <property name="url"
value="jdbc:mysql://localhost:3306/springtest?characterEncoding=utf8" />
    <property name="username" value="root" />
    <property name="password" value="root" />
    <!-- 最大连接数 -->
    <property name="maxTotal" value="30"/>
    <!-- 最大空闲连接数 -->
    <property name="maxIdle" value="10"/>
    <!-- 初始化连接数 -->
    <property name="initialSize" value="5"/>
</bean>
<!-- 添加事务支持 -->
<bean id="txManager"
    class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource" />
</bean>
<!-- 开启事务注解-->
<tx:annotation-driven transaction-manager="txManager" />
<!-- 配置 MyBatis 工厂，同时指定数据源，并与 MyBatis 完美整合 -->
<bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactoryBean">
    <property name="dataSource" ref="dataSource" />
    <!-- configLocation 的属性值为 MyBatis 的核心配置文件 -->
    <property name="configLocation" value="classpath:com/mybatis/mybatis-config.xml"/>
</bean>
<!--Mapper 代理开发，使用 Spring 自动扫描 MyBatis 的接口并装配
（Spring 将指定包中所有被@Mapper 注解标注的接口自动装配为 MyBatis 的映射接口） -->
<bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
    <!-- mybatis-spring 组件的扫描器 -->
    <property name="basePackage" value="com.dao"/>
    <property name="sqlSessionFactoryBeanName" value="sqlSessionFactory"/>
</bean>

```

</beans>

10、创建测试类

在包 `com.controller` 中，创建测试类 `TestController`，代码如下：

```
package com.controller;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class TestController {

    public static void main(String[] args) {

        ApplicationContext appCon = new ClassPathXmlApplicationContext("applicationContext.xml");
        UserController uc = (UserController)appCon.getBean("userController");
        uc.test();

    }

}
```

11、测试并运行

测试并运行 Web 应用 `practice4`。

Java EE框架整合开发实验指导书 陈恒编写

实验五 Controller 接收请求参数（注册与登录系统）

（实验课时：2 实验性质：设计）

实验名称：

Controller 接收请求参数（注册与登录系统）

实验目的和要求：

- 1、掌握 Controller 接收请求参数的方式。
- 2、掌握 Spring MVC 的重定向和转发的实现方法。
- 3、掌握 RequestMapping 注解的用法。

实验内容和步骤：

- 1、创建 Web 应用并导入相关的 JAR 包

创建 Web 应用 practice5，并导入相关 JAR 包，包括实验四的 Web 应用 practice4 中的 JAR 包以及 Spring MVC 所需要的 JAR 包。

- 2、创建日志文件

将 practice4 的日志文件复制到 practice5 的对应位置。

- 3、创建 Web 应用的页面

该应用共涉及 4 个 JSP 页面，分别为 index.jsp、login.jsp、register.jsp 以及 main.jsp。单击 index.jsp 中的“去注册”超链接打开 register.jsp，单击 index.jsp 中的“去登录”超链接打开 login.jsp。注册成功跳转到 login.jsp，登录成功跳转到 main.jsp。

在应用 practice5 的 WebContent 目录下创建 index.jsp 页面，运行效果如图 5 所示。

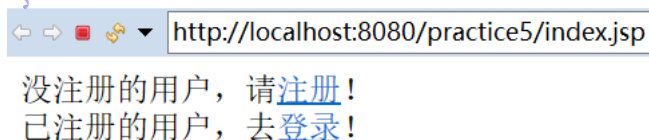


图 5 index.jsp

在 WEB-INF 目录下，创建 pages 目录，并在该目录下创建 login.jsp、register.jsp 以及 main.jsp。

login.jsp 的运行效果如图 6 所示。



图 6 login.jsp

register.jsp 的运行效果如图 7 所示。

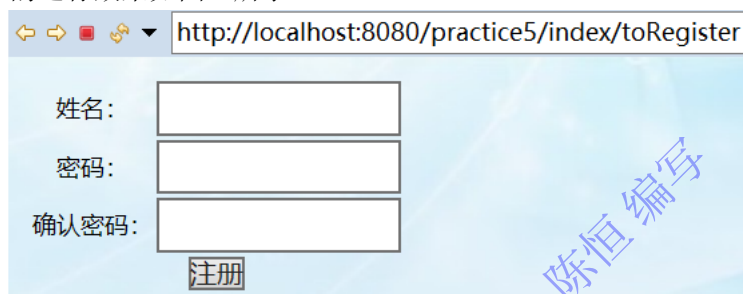


图 7 register.jsp

main.jsp 的运行效果如图 8 所示。

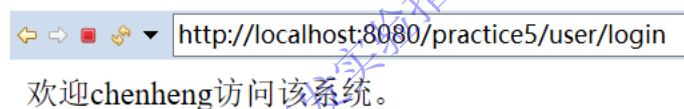


图 8 main.jsp

4、创建数据表及实体类

在数据库 springtest 创建数据表 usertable（如图 9 所示）及实体类 UserTable。

位	索引	外键	触发器	选项	注释	SQL 预览
名					类型	长度
id					int	11
uname					varchar	50
upwd					varchar	20
					小数点	允许空值 (
						1

图 9 表 usertable

创建名为 com.entity 的包，并在该包中创建实体类 UserTable。实体类 UserTable 的代码略。

5、创建数据访问接口

创建名为 com.dao 的包，并在该包下创建数据访问接口 UserTableDao。该接口中有两个接口方法，一个对应注册功能，一个对应登录功能。

UserTableDao 的代码如下：

```
package com.dao;
```

```
import java.util.List;
import org.springframework.stereotype.Repository;
import com.entity.UserTable;
@Repository
public interface UserTableDao {
    int register(UserTable user);
    List<UserTable> login(UserTable user);
}
```

6、创建映射文件及 MyBatis 配置文件

创建名为 com.mybatis 的包，并在该包下创建 SQL 映射文件 UserTableMapper.xml 和 MyBatis 配置文件 mybatis-config.xml。

UserTableMapper.xml 的代码如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org/DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.dao.UserTableDao">
    <select id="login" parameterType="com.entity.UserTable"
        resultType="com.entity.UserTable">
        select * from usertable where uname = #{uname} and upwd = #{upwd}
    </select>
    <insert id="register" parameterType="com.entity.UserTable">
        insert into usertable (id,uname,upwd) values(null, #{uname},#{upwd})
    </insert>
</mapper>
```

mybatis-config.xml 的代码如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
PUBLIC "-//mybatis.org/DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <!-- 告诉 MyBatis 到哪里去找映射文件 -->
```

```

    <mappers>

        <mapper resource="com/mybatis/UserTableMapper.xml"/>

    </mappers>

</configuration>

```

7、创建控制器层

创建名为 `com.controller` 的包，并在该包下创建两个控制器类。一个处理首页的超链接请求 `IndexController.java`，一个处理注册与登录请求 `UserController.java`（在该类依赖注入数据访问层）。

8、创建 Spring MVC 的配置文件及 web.xml

在 `WEB-INF` 目录下，Spring MVC 的配置文件 `springmvc-servlet.xml`，以及 `web.xml` 配置文件。

`springmvc-servlet.xml` 的内容如下：

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd
           http://www.springframework.org/schema/mvc
           http://www.springframework.org/schema/mvc/spring-mvc.xsd">
    <!-- 配置数据源 -->
    <bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="com.mysql.jdbc.Driver" />
        <property name="url" value="jdbc:mysql://localhost:3306/springtest?characterEncoding=utf8" />
        <property name="username" value="root" />
        <property name="password" value="root" />
    <!-- 最大连接数 -->

```

```

        <property name="maxTotal" value="30"/>

        <!-- 最大空闲连接数 -->

        <property name="maxIdle" value="10"/>

        <!-- 初始化连接数 -->

        <property name="initialSize" value="5"/>

    </bean>

    <!-- 配置 MyBatis 工厂，同时指定数据源，并与 MyBatis 完美整合 -->

    <bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactoryBean">

        <property name="dataSource" ref="dataSource" />

        <!-- configLocation 的属性值为 MyBatis 的核心配置文件 -->

        <property name="configLocation" value="classpath:com/mybatis/mybatis-config.xml"/>
    </bean>

    <!--Mapper 代理开发，使用 Spring 自动扫描 MyBatis 的接口并装配
    （Spring 将指定包中所有被@Mapper 注解标注的接口自动装配为 MyBatis 的映射接口） -->

    <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">

        <!-- mybatis-spring 组件的扫描器 -->

        <property name="basePackage" value="com.dao"/>

        <property name="sqlSessionFactoryBeanName" value="sqlSessionFactory"/>
    </bean>

    <!-- 使用扫描机制，扫描控制器类 -->

    <context:component-scan base-package="com.controller"/>

    <mvc:annotation-driven />

    <!--允许 images 目录下所有文件可见 -->

    <mvc:resources location="/images/" mapping="/images/**"></mvc:resources>

    <!-- 配置视图解析器 -->

    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver"

        id="internalResourceViewResolver">

        <!-- 前缀 -->

        <property name="prefix" value="/WEB-INF/pages/" />

        <!-- 后缀 -->

        <property name="suffix" value=".jsp" />
    </bean>

</beans>

```

web.xml 的内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xmlns="http://xmlns.jcp.org/xml/ns/javaee"

xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee

http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"

id="WebApp_ID" version="3.1">

<!--部署 DispatcherServlet-->

<servlet>

    <servlet-name>springmvc</servlet-name>

    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

    <!-- 表示容器在启动时立即加载 servlet -->

    <load-on-startup>1</load-on-startup>

</servlet>

<servlet-mapping>

    <servlet-name>springmvc</servlet-name>

    <!-- 处理所有 URL-->

    <url-pattern>/</url-pattern>

</servlet-mapping>

</web-app>
```

9、测试运行

运行主页 index.jsp，进行注册与登录功能的测试。

实验六 表单与数据绑定（学生信息管理项目）

（实验课时：2 实验性质：设计）

实验名称：

表单与数据绑定（学生信息管理项目）

实验目的和要求：

- 1、理解数据绑定的基本原理，掌握表单标签库的用法。
- 2、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

- 1、创建 Web 应用并导入相关的 JAR 包

创建 Web 应用 practice6，并导入相关 JAR 包，包括实验五的 Web 应用 practice5 中的 JAR 包、Spring MVC 所需要的 JAR 包以及 JSTL 所需要的包 taglibs-standard-impl-1.2.5.jar 和 taglibs-standard-spec-1.2.5.jar。

- 2、创建日志文件

将 practice5 的日志文件复制到 practice6 的对应位置。

- 3、创建 Web 应用的页面

该应用共涉及 2 个 JSP 页面，分别为 addInput.jsp 和 result.jsp。在 addInput.jsp 页面输入学生的基本信息后单击“添加”按钮，添加成功后跳转到 result.jsp 显示所有学生信息。

在 WEB-INF 目录下，创建 pages 目录，并在该目录下创建 addInput.jsp（参考教材的 12.3 节使用 Spring 的表单标签实现该页面）和 result.jsp。

addInput.jsp 的运行效果如图 10 所示。

图 10 addInput.jsp

result.jsp 的运行效果如图 11 所示。

学号	姓名	性别	年龄	院系
20191111	陈恒	男	88	计算机学院
20191112	chenheng	男	99	马克思主义学院
20191113	陈恒333	男	77	英语学院

图 11 result.jsp

4、创建数据表及实体类

在数据库 springtest 创建数据表 studenttable（如图 12 所示）及实体类 StudentTable。

名	类型	长度	小数点	允许空值 (	
id	int	11	0	☐	1
sno	varchar	20	0	☒	
sname	varchar	20	0	☒	
sex	varchar	2	0	☒	
age	int	2	0	☒	
dept	varchar	10	0	☒	

图 12 表 studenttable

创建名为 com.entity 的包，并在该包中创建实体类 StudentTable。实体类 StudentTable 的代码略。

5、创建数据访问接口

创建名为 `com.dao` 的包，并在该包下创建数据访问接口 `StudentTableDao`。该接口中有两个接口方法，一个对应添加学生功能，一个对应查询所有学生功能。

`StudentTableDao` 的代码如下：

```
package com.dao;
import java.util.List;
import org.springframework.stereotype.Repository;
import com.entity.StudentTable;
@Repository
public interface StudentTableDao {
    int addStudent(StudentTable student);
    List<StudentTable> allStudent();
}
```

6、创建映射文件及 MyBatis 配置文件

创建名为 `com.mybatis` 的包，并在该包下创建 SQL 映射文件 `StudentTableMapper.xml` 和 MyBatis 配置文件 `mybatis-config.xml`。

`StudentTableMapper.xml` 的代码如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.dao.StudentTableDao">
    <select id="allStudent" resultType="StudentTable">
        select * from studenttable
    </select>
    <insert id="addStudent" parameterType="StudentTable">
        insert into studenttable (id,sno,sname,sex,age,dept)
        values(null, #{sno},#{sname},#{sex},#{age},#{dept})
    </insert>
</mapper>
```

`mybatis-config.xml` 的代码如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <!-- 告诉 MyBatis 到哪里去找映射文件 -->
    <typeAliases>
        <!-- 给类型 com.entity.StudentTable 定义别名 StudentTable-->
        <typeAlias type="com.entity.StudentTable" alias="StudentTable"/>
    </typeAliases>
</configuration>
```

```

</typeAliases>
<mappers>
    <mapper resource="com/mybatis/StudentTableMapper.xml"/>
</mappers>
</configuration>

```

7、创建 Service 层

创建名为 `com.service` 的包，并在该包中创建 `StudentService` 接口和接口实现类 `StudentServiceImpl`（在该类中依赖注入数据访问层）。

`StudentService` 接口的代码如下：

```

package com.service;
import org.springframework.ui.Model;
import com.entity.StudentTable;
public interface StudentService {
    public String toAdd(Model model);
    public String add(Model model, StudentTable student);
    public String allStudent(Model model);
}

```

`StudentServiceImpl` 类的代码如下：

```

package com.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.ui.Model;
import com.dao.StudentTableDao;
import com.entity.StudentTable;
@Service
public class StudentServiceImpl implements StudentService{
    @Autowired
    private StudentTableDao studentTableDao;
    @Override
    public String toAdd(Model model) {
        String dept[] = { "日语学院", "计算机学院", "英语学院", "马克思主义学院", "其它" };
        model.addAttribute("dept", dept);
        StudentTable student = new StudentTable();
        student.setSex("男");//默认选中
        model.addAttribute("student", student);
        return "addInput";
    }
    @Override
    public String add(Model model, StudentTable student) {
        if(student.getAge() < 18) { //添加失败

```

```

        String dept[] = { "日语学院", "计算机学院", "英语学院", "马克思主义学院", "其它" };
        model.addAttribute("dept", dept);
        return "addInput";
    } else {
        studentTableDao.addStudent(student);
        return "forward:/student/allStudent";//回到查询的控制器方法
    }
}

@Override
public String allStudent(Model model) {
    model.addAttribute("allStus", studentTableDao.allStudent());
    return "result";
}
}

```

8、创建控制器层

创建名为 `com.controller` 的包，并在该包中创建控制器类 `StudentController`（在该类中依赖注入 `Service` 层），该控制器类中有三个方法（调用 `Service` 层的方法），一个是打开 `addInput.jsp`，一个是实现添加学生功能，另一个是实现查询所有学生功能。

9、创建 Spring MVC 的配置文件及 `web.xml`

在 `WEB-INF` 目录下，Spring MVC 的配置文件 `springmvc-servlet.xml`，以及 `web.xml` 配置文件。

`springmvc-servlet.xml` 的内容如下：

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd">
    <!-- 配置数据源 -->
    <bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="com.mysql.jdbc.Driver" />
        <property name="url" value="jdbc:mysql://localhost:3306/springtest?characterEncoding=utf8" />
        <property name="username" value="root" />
    </bean>

```

```

        <property name="password" value="root" />
        <!-- 最大连接数 -->
        <property name="maxTotal" value="30"/>
        <!-- 最大空闲连接数 -->
        <property name="maxIdle" value="10"/>
        <!-- 初始化连接数 -->
        <property name="initialSize" value="5"/>
    </bean>
    <!-- 配置 MyBatis 工厂，同时指定数据源，并与 MyBatis 完美整合 -->
    <bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactoryBean">
        <property name="dataSource" ref="dataSource" />
        <!-- configLocation 的属性值为 MyBatis 的核心配置文件 -->
        <property name="configLocation" value="classpath:com/mybatis/mybatis-config.xml"/>
    </bean>
    <!--Mapper 代理开发，使用 Spring 自动扫描 MyBatis 的接口并装配
    （Spring 将指定包中所有被@Mapper 注解标注的接口自动装配为 MyBatis 的映射接口） -->
    <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
        <!-- mybatis-spring 组件的扫描器 -->
        <property name="basePackage" value="com.dao"/>
        <property name="sqlSessionFactoryBeanName" value="sqlSessionFactory"/>
    </bean>
    <!-- 使用扫描机制，扫描控制器层 -->
    <context:component-scan base-package="com.controller"/>
    <!-- 使用扫描机制，扫描 Service 层 -->
    <context:component-scan base-package="com.service"/>
    <mvc:annotation-driven />
    <!--允许 images 目录下所有文件可见 -->
    <mvc:resources location="/images/" mapping="/images/**"></mvc:resources>
    <!-- 配置视图解析器 -->
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver"
        id="internalResourceViewResolver">
        <!-- 前缀 -->
        <property name="prefix" value="/WEB-INF/pages/" />
        <!-- 后缀 -->
        <property name="suffix" value=".jsp" />
    </bean>
</beans>

```

web.xml 的内容如下：

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee

```

```
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  id="WebApp_ID" version="3.1">
    <!--部署 DispatcherServlet-->
    <servlet>
      <servlet-name>springmvc</servlet-name>
      <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
      <!-- 表示容器在启动时立即加载 servlet -->
      <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
      <servlet-name>springmvc</servlet-name>
      <!-- 处理所有 URL-->
      <url-pattern>/</url-pattern>
    </servlet-mapping>
    <!-- 避免中文乱码 -->
    <filter>
      <filter-name>characterEncodingFilter</filter-name>
      <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
      <init-param>
        <param-name>encoding</param-name>
        <param-value>UTF-8</param-value>
      </init-param>
      <init-param>
        <param-name>forceEncoding</param-name>
        <param-value>true</param-value>
      </init-param>
    </filter>
    <filter-mapping>
      <filter-name>characterEncodingFilter</filter-name>
      <url-pattern>/*</url-pattern>
    </filter-mapping>
  </web-app>
```

10、测试运行

发布并启动 tomcat，通过 <http://localhost:8080/practice6/student/toAdd> 运行添加学生页面 addInput.jsp，进行添加与查询功能的测试。

实验七 拦截器应用案例（登录权限控制）

（实验课时：2 实验性质：设计）

实验名称：

拦截器应用案例（登录权限控制）

实验目的和要求：

- 1、理解拦截器的原理，掌握拦截器的定义与配置。
- 2、掌握拦截器的实际应用。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

本实验将通过拦截器来完成一个用户登录权限验证的 Web 应用 practice7。要求：只有成功登录的用户才能访问系统的主页面 main.jsp，如果没有成功登录而直接访问主页面，则拦截器将请求拦截，并转发到登录页面 login.jsp。当成功登录的用户在系统主页面中单击“退出”链接时，回到登录页面。具体实现步骤见教材的 13.3 节。

实验八 JSR 303 验证（表单验证）

（实验课时：2 实验性质：设计）

实验名称：

JSR 303 验证（表单验证）

实验目的和要求：

- 1、掌握使用 JSR 303（Java 验证规范）对表单数据进行验证。
- 2、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

本节使用一个应用 practice8 讲解 JSR 303 验证的编写及使用。该应用中有 1 个数据输入页面 addGoods.jsp，效果如图 13 所示；有 1 个数据显示页面 goodsList.jsp，效果如图 14 所示。



图 13 数据输入页面

商品名	商品详情	商品价格	创建日期
鸡蛋	好鸡蛋	8.8	Sat Feb 03 00:00:00 CST 2018

图 14 数据显示页面

验证要求如下：

- 1、商品名和商品详情不能为空。
- 2、商品价格在 0-100 之间。
- 3、创建日期不能在系统日期之后。

根据上述要求，参考教材的 14.2.3 节和 14.3.3 节完成应用 practice8。

实验九 SSM 框架整合应用测试（学生信息管理项目）

（实验课时：2 实验性质：设计）

实验名称：

SSM 框架整合应用测试（学生信息管理项目）

实验目的和要求：

- 1、了解 SSM 框架整合思路。
- 2、掌握 SSM 框架整合环境构建。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

使用 SSM 框架实现学生信息管理项目 practice9。practice9 共涉及 2 个 JSP 页面，分别为 addInput.jsp 和 result.jsp。在 addInput.jsp 页面输入学生的基本信息后单击“添加”按钮，添加成功后跳转到 result.jsp 显示所有学生信息。具体实验步骤参考教材的 19.2 节。

实验十 在 Eclipse（或 STS 或 IDEA）中使用 Maven 整合 SSM 框架 （学生信息管理项目）

（实验课时：2 实验性质：设计）

实验名称：

在 Eclipse（或 STS 或 IDEA）中使用 Maven 整合 SSM 框架（学生信息管理项目）

实验目的和要求：

- 1、掌握 Maven Web 应用的创建过程。
- 2、掌握使用 Maven 管理 SSM 框架所需要的 JAR 包。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

参考教材的附录 3，使用 Maven 管理 SSM 框架所需要的 JAR 包，实现学生信息管理项目 practice10（即把 practice9 的 JAR 包使用 Maven 管理）。pom.xml 的内容如下：

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.chenheng</groupId>
  <artifactId>practice10</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>test01 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <properties>
    <spring.version>5.1.9.RELEASE</spring.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-core</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${spring.version}</version>
    </dependency>
  </dependencies>
</project>
```

```
<groupId>org.springframework</groupId>
<artifactId>spring-beans</artifactId>
<version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-expression</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.45</version>
</dependency>
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-dbcp2</artifactId>
    <version>2.7.0</version>
</dependency>
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis</artifactId>
    <version>3.5.2</version>
</dependency>
<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>
```

```
<dependency>
  <groupId>org.mybatis</groupId>
  <artifactId>mybatis-spring</artifactId>
  <version>2.0.2</version>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>

</dependencies>
<build>
  <finalName>test01</finalName>
</build>
</project>
```

Java EE框架整合开发实验指导书 陈恒编写

实验十一 综合实验（电子商务平台的设计与实现）

（实验课时：16 实验性质：设计）

实验名称：

综合实验（电子商务平台的设计与实现）

实验目的和要求：

- 1、掌握 SSM 框架应用开发的流程、方法以及技术。
- 2、熟悉电子商务平台的业务需求、设计以及实现。
- 3、认真书写实验报告，如实填写各项实验内容。

实验内容和步骤：

具体实验内容和步骤，见教材第 20 章。

Java EE 框架整合开发实验指导书 陈恒编写